WORKSHOP HANDOUT

Autonomous Agents

OpenClaw & ZeroClaw — Hands-On für Entwickler

OpenClaw & ZeroClaw — Developer Hands-On Guide

🐾

OpenClaw

Node.js · Cloud-Modelle · Multi-Agent Gateway
Läuft lokal oder in der Cloud

🦀

ZeroClaw

Rust · 3MB Binary · 65 Provider · Hardware-Support
Läuft lokal oder in der Cloud

📅 2026 👤 Oskar · execute008 🌐 openclaw.ai · zeroclawlabs.ai

Was ist ein autonomer Agent?

GRUNDLAGEN / BASICS

Ein **autonomer Agent** ist ein KI-System, das nicht nur antwortet — es *handelt*. Es kann Werkzeuge verwenden, Entscheidungen treffen, Aufgaben sequentiell ausführen und proaktiv auf seine Umgebung reagieren.

*An **autonomous agent** doesn't just respond — it **acts**. Tools, decisions, sequences — and it can initiate on its own.*

Chatbot vs. Agent

❌ Klassischer Chatbot

- Fragt → bekommt Antwort
- Kein Gedächtnis zwischen Sessions
- Kein Zugriff auf externe Systeme
- Rein reaktiv, immer on-demand

✅ Autonomer Agent

- Empfängt Ziel → plant → führt aus
- Persistentes Gedächtnis (Dateien/SQLite)
- Tools: Browser, Shell, APIs, Dateien
- Proaktiv: arbeitet ohne User-Input

Die Agent-Schleife / The Agent Loop

👁️ **Wahrnehmen** Perceive → 🧠 **Denken** Reason → 🔧 **Handeln** Act (Tools) → 💾 **Merken** Remember → 🔄 **Nächster Turn** Next cycle

Werkzeuge / Tools (Beispiele)

🌐 **web_search / web_fetch**
Internet-Recherche in Echtzeit

💻 **exec / shell**
Befehle auf dem Host ausführen

📁 **read / write / edit**
Dateisystem-Operationen

🌍 **browser**
Echten Browser steuern (Playwright)

💬 **message**
Telegram, Discord, Slack senden

🧠 **memory_search**
Semantische Suche im Gedächtnis

Memory-System

Agenten sind von Natur aus zustandslos — **Memory-Dateien sind das Persistenz-Layer**:

- `MEMORY.md` / `memory.sqlite` — Langzeit-Gedächtnis (kuratiert)
- `memory/YYYY-MM-DD.md` — Tagesnotizen, rohe Logs
- `SOUL.md` — Persönlichkeit & Werte des Agenten
- `USER.md` — Kontext über den Menschen
- `HEARTBEAT.md` — Periodische Aufgaben-Checkliste

> 💡 **Key Insight**
> Das LLM ist das Gehirn — die Tools sind die Hände — die Memory-Dateien sind das Langzeitgedächtnis. Erst zusammen entsteht echter Nutzen.

OpenClaw vs ZeroClaw

VERGLEICH / COMPARISON

Beide Plattformen sind **vollständige Agent-Runtimes** — der Unterschied liegt nicht darin *wo* sie laufen, sondern in der **Architektur und dem Technologie-Stack**. Beide können lokal *und* in der Cloud betrieben werden.

> ⚠️ **Wichtiger Hinweis / Important Note**
> OpenClaw läuft nicht nur in der Cloud — auch ZeroClaw läuft nicht nur lokal.
> **Beide können beides.**
> Die Wahl hängt von Stack, Datenschutz-Anforderungen und Modell-Präferenz ab.

Feature	🐾 **OpenClaw** (Node.js)	🦀 **ZeroClaw** (Rust)
Stack	Node.js / TypeScript	Rust — 3MB Binary, kein Runtime
Deployment	✓ **Lokal + Cloud + Server**	✓ **Lokal + Cloud + Server + Android**
Provider	~10 Provider (Anthropic, OpenAI, Ollama...)	**65 Provider** inkl. Ollama, LM Studio, vLLM, DeepSeek...
100% lokal möglich	✓ **mit Ollama**	✓ **mit Ollama/LM Studio/llama.cpp**
Channels	Telegram, Discord, Signal, WhatsApp, Slack, iMessage...	20+ Channels inkl. Nostr, Matrix, IRC, DingTalk, Lark, Reddit, Bluesky
Hardware-Support	—	✓ **STM32, Raspberry Pi GPIO, USB peripherals**
Skills / Extensions	ClawHub Marketplace (npm)	Built-in skill-creation Cargo feature
Multi-Agent	✓ **MQTT, Gateway, subagents**	✓ **Daemon + WebSocket Gateway**
Memory	Markdown-Dateien	SQLite (auto-save) + Markdown
Migration	—	✓ `zeroclaw migrate openclaw`
Autonomy Control	Tool policies, approval flows	E-Stop, OTP, max actions/hour, cost/day limit
Resource footprint	Node.js + npm packages	3MB Binary, kein Node, kein Python

Wann was? / Decision Guide

🐾 Nimm OpenClaw wenn...

- Du im JS/TS-Ökosystem bleiben willst
- ClawHub Skills nutzen/publizieren willst
- Multi-Agent mit MQTT + Subagents
- Deep Telegram/Discord integration
- ACP Coding-Agent Workflows

🦀 Nimm ZeroClaw wenn...

- Minimaler Footprint wichtig ist
- Hardware steuern (GPIO, STM32)
- Maximale Provider-Auswahl (65)
- Niche Channels (Nostr, Matrix, IRC...)
- Migration von OpenClaw geplant

ZeroClaw — Deep Dive

ZEROCLAW FEATURES

Installation (eine Zeile)

```
# Interaktiv (recommended)
curl -fsSL https://zeroclawlabs.ai/install.sh | bash

# Non-interaktiv mit Ollama (kein API Key!)
./install.sh --provider ollama --prefer-prebuilt

# Non-interaktiv mit Anthropic
./install.sh --api-key "sk-ant-..." --provider anthropic
```

65 Provider — Auswahl

☁️ Cloud

OpenRouter · Anthropic · OpenAI · Gemini · Groq · Mistral · DeepSeek · xAI · Bedrock · Cohere · Perplexity + 40 weitere

🖥️ Lokal (kein API Key)

Ollama · LM Studio · llama.cpp · vLLM · SGLang · Claude Code CLI · Gemini CLI · Osaurus

🔧 Custom

`custom:<URL>`
Jeder OpenAI-kompatible Endpoint

`anthropic-custom:<URL>`
Jeder Anthropic-kompatibler Endpoint

Config (`~/.zeroclaw/config.toml`)

```
# Lokal — komplett offline, kein API Key
default_provider = "ollama"
default_model    = "qwen3:8b"

# Cloud — maximale Power
default_provider = "anthropic"
api_key          = "sk-ant-..."
default_model    = "claude-opus-4"

# Server-Deploy — VPS mit Ollama
default_provider  = "ollama"
gateway_port      = 18789
gateway_host      = "0.0.0.0"
```

Daemon (Always-On auf Server/Laptop)

```
# Als Daemon starten (Gateway + Channels + Heartbeat)
zeroclaw daemon

# Als systemd User Service registrieren
zeroclaw service install

# Status prüfen
zeroclaw status
zeroclaw doctor
```

Autonomy & Sicherheit

🛡️ Safety Controls

- **E-Stop** — Notfall-Stopp (sofort)
- **OTP** — Einmal-Passwort für sensitive Actions
- **max_actions_per_hour** — Rate limit
- **max_cost_per_day** — Budget limit ($)
- **allowed_commands** — Shell-Whitelist
- **workspace_only** — Dateisystem-Sandbox

🔩 Hardware (einzigartig!)

- USB device discovery
- STM32 microcontroller support
- Raspberry Pi GPIO control
- `zeroclaw hardware` — scan
- `zeroclaw peripheral` — manage
- IoT / embedded agents möglich

Migration von OpenClaw

```
# OpenClaw-Workspace in ZeroClaw importieren
zeroclaw migrate openclaw

# Memory aus openclaw.json importieren
# Alle .md Dateien werden übernommen
```

✨ Echtzeit-Demo heute

ZeroClaw läuft auf diesem Server mit Ollama/qwen3:8b — komplett lokal, kein API Key. Getestet: `zeroclaw agent -m "Hello"` → antwortet in ~1s.

Skills & Multi-Agent-Setups

SKILLS / MULTI-AGENT

Skills — Modulare Erweiterungen

Skills sind Ordner mit einer `SKILL.md` — sie erklären dem Agenten neue Fähigkeiten. Kein Code-Deploy, nur Text.

```
my-skill/
├── SKILL.md          # Trigger-Bedingungen + Anleitung für den Agenten
├── references/       # API-Specs, Docs
└── scripts/          # Hilfs-Skripte
```

OpenClaw: `npx clawhub install weather` — aus dem ClawHub Marketplace
ZeroClaw: `zeroclaw skills install` / manuell in `~/.zeroclaw/skills/`

🌤️ **weather**
Wetter via wttr.in / Open-Meteo (kein Key)

📝 **notion**
Notion Pages, Datenbanken, Blocks

💻 **coding-agent**
Codex / Claude Code als Subagent

🎤 **whisper-api**
Audio-Transkription via OpenAI

Multi-Agent — Kommunikationsarchitektur

📡 **MQTT (OpenClaw)**

- Broker: Mosquitto (Podman)
- Topics: `hive/agent/inbox`
- Push-basiert, event-driven
- Agents auf verschiedenen Maschinen
- Chat-Inject in laufende Session

🔗 **Gateway/WebSocket (ZeroClaw)**

- Daemon öffnet HTTP + WS Server
- `ws://host:18789/ws/chat`
- Subagents via `sessions_spawn()`
- ACP-Protokoll für Coding-Agents
- REST API für externe Trigger

🐉 **Reales Setup: Draco (OpenClaw) + ZeroClaw Demo (heute)**

Draco läuft als OpenClaw auf diesem Server (dr460nd3n, 24/7). ZeroClaw läuft parallel auf demselben Server mit Ollama. Beide können über MQTT oder HTTP kommunizieren.

```
Draco (OpenClaw, Port 18789)  ←──→  ZeroClaw (Port 3000)
        ↕ MQTT: hive/draco/inbox          ↕ ws://localhost:3000/ws/chat
   Telegram / Discord / Signal       Nostr / Matrix / IRC / Bluesky
```

Design-Prinzipien für Multi-Agent

- **Separation of concerns:** Jeder Agent hat eine klare Rolle
- **Skepticism:** Agents sollen sich gegenseitig nicht blind vertrauen
- **Async by default:** Push-Kommunikation statt Polling
- **Memory isolation:** Private Daten bleiben beim Owner-Agent

- **Human-in-the-loop:** Für destructive Actions immer Bestätigung

Quickstart & Ressourcen

SETUP / RESOURCES

🐾 OpenClaw in 3 Schritten

```
npm install -g openclaw

openclaw configure
# → Provider, API Key, Channels

openclaw gateway start
openclaw status
```

Voraussetzungen

Node.js ≥ 18, API Key (z.B. Anthropic), optional: Telegram Bot Token

🦀 ZeroClaw in 1 Schritt

```
curl -fsSL \
  https://zeroclawlabs.ai/install.sh \
  | bash

# Lokal ohne API Key:
zeroclaw onboard --provider ollama

zeroclaw daemon
zeroclaw status
```

Voraussetzungen

Nichts (Rust wird mitinstalliert). Für lokale Modelle: Ollama.

Workspace-Struktur (beide Plattformen)

```
~/.openclaw/workspace/    OR    ~/.zeroclaw/workspace/
├── SOUL.md        # Persönlichkeit des Agenten
├── USER.md        # Kontext über den Menschen
├── MEMORY.md      # Langzeitgedächtnis (kuratiert)
├── AGENTS.md      # Workspace-Konventionen
├── HEARTBEAT.md   # Periodische Aufgaben-Checkliste
├── TOOLS.md       # Umgebungs-spezifische Notizen (SSH, Cameras…)
└── memory/
    └── YYYY-MM-DD.md   # Tages-Logs
```

Ressourcen / Resources

OPENCLAW DOCS
docs.openclaw.ai

ZEROCLAW
zeroclawlabs.ai

SKILL REGISTRY
clawhub.com

COMMUNITY DISCORD
discord.com/invite/clawd

WORKSHOP REPO
github.com/execute008/autonomous-agents-workshop

ZEROCLAW NPM
npmjs.com/package/zeroclaw

🚀 **Take-Away**

OpenClaw und ZeroClaw lösen dasselbe Problem auf zwei Arten. OpenClaw ist das JS-Ökosystem mit ClawHub und ACP. ZeroClaw ist das Rust-Binary mit 65 Providern und Hardware-Support. Beide können lokal *und* in der Cloud laufen, beide unterstützen Ollama für komplette Datenprivacy.

The real shift: you give the LLM hands, memory, and a persistent identity. That's when it stops being a chatbot and starts being an agent.